

PR Audit Report

Repository	dipak-shaaki/dummy-audit-test
PR Number	#4
Risk Level	HIGH

PR Audit Report

Summary

This PR appears to replace a custom audit workflow with a Bandit-based security scan and adds several demo files containing intentionally vulnerable patterns and hard-coded secrets.

I only reported issues that were explicitly surfaced by the provided SonarCloud or Ruff findings. Bandit reported no findings, and there are no Dependabot alerts.

What the Developer Was Trying to Do

The developer seems to be:

- simplifying the GitHub Actions workflow to run a security scan on pull requests,
- adding example code that demonstrates common security anti-patterns,
- and adding configuration values for a demo environment.

Security Issues (Bandit + Dependabot + SonarCloud)

1) Hard-coded credential in `app.py`

Finding: SonarCloud python:S2068 — hard-coded credential detected.

Full function containing the issue:

```
import os
import subprocess
import pickle
import yaml
import hashlib
import logging
import sqlite3

# === HARD CODED SECRETS (Secret Scanning will catch these) ===
SECRET_KEY = "supersecret_hardcoded_key_12345"
DB_PASSWORD = "P@ssw0rd2026!"
AWS_ACCESS_KEY = "AKIAIOSFODNN7EXAMPLE"
API_TOKEN = "ghp_abcdefghijklmnopqrstuvwxyz123456"

def run_command(user_input):
    # Command Injection
    os.system("ls " + user_input) # Dangerous
    subprocess.call("echo " + user_input, shell=True)
```

Problematic line:

```
SECRET_KEY = "supersecret_hardcoded_key_12345" # <- ISSUE HERE
```

Why this is problematic / risk:

Hard-coded secrets can be exposed through source control, logs, forks, or accidental reuse. If this value is real, it should be treated as compromised and rotated immediately.

Corrected version of the full function/module section:

```
import os
import subprocess
import pickle
import yaml
import hashlib
import logging
import sqlite3
import random
```

```
SECRET_KEY = os.environ.get("SECRET_KEY")

def run_command(user_input):
    # Command Injection
    os.system("ls " + user_input) # Dangerous
    subprocess.call("echo " + user_input, shell=True)
```

Official documentation:

- Sonar rule: <https://rules.sonarsource.com/python/RSPEC-2068/>
-

2) Hard-coded PostgreSQL password in `config/config.py`

Finding: SonarCloud secrets: S6698 — hard-coded PostgreSQL password.

Full function containing the issue:

```
DATABASE_URL = "postgresql://user:SuperSecretPass123@db.example.com/prod"
DEBUG = True
SECRET = "dev-secret-123456789"
```

Problematic line:

```
DATABASE_URL = "postgresql://user:SuperSecretPass123@db.example.com/prod" # <-
ISSUE HERE
```

Why this is problematic / risk:

Embedding database credentials in source code exposes them to anyone with repository access and increases the chance of credential leakage. This should be moved to environment variables or a secret manager.

Corrected version of the full function/module section:

```
import os

DATABASE_URL = os.environ.get("DATABASE_URL")
DEBUG = True
SECRET = os.environ.get("SECRET")
```

Official documentation:

- Sonar rule: <https://rules.sonarsource.com/python/RSPEC-6698/>
-

3) Hard-coded secret in `config/config.py`

Finding: SonarCloud python:S6418 — hard-coded secret detected.

Full function containing the issue:

```
DATABASE_URL = "postgresql://user:SuperSecretPass123@db.example.com/prod"
DEBUG = True
SECRET = "dev-secret-123456789"
```

Problematic line:

```
SECRET = "dev-secret-123456789" # <- ISSUE HERE
```

Why this is problematic / risk:

A value named SECRET is especially likely to be sensitive. Keeping it in code makes it easy to leak and difficult to rotate safely.

Corrected version of the full function/module section:

```
import os

DATABASE_URL = os.environ.get("DATABASE_URL")
DEBUG = True
SECRET = os.environ.get("SECRET")
```

Official documentation:

- Sonar rule: <https://rules.sonarsource.com/python/RSPEC-6418/>

Code Quality Issues (Ruff + SonarCloud)

1) Syntax error in `app.py` at the `run\_command` section

Finding: Ruff invalid-syntax — unexpected indentation.

Full function containing the issue:

```
import os
import subprocess
import pickle
import yaml
import hashlib
import logging
```

```
import sqlite3

# === HARD CODED SECRETS (Secret Scanning will catch these) ===
SECRET_KEY = "supersecret_hardcoded_key_12345"
DB_PASSWORD = "P@ssw0rd2026!"
AWS_ACCESS_KEY = "AKIAIOSFODNN7EXAMPLE"
API_TOKEN = "ghp_abcdefghijklmnopqrstuvwxyz123456"

def run_command(user_input):
    # Command Injection
    os.system("ls " + user_input) # Dangerous
    subprocess.call("echo " + user_input, shell=True)
```

Problematic line:

```
os.system("ls " + user_input) # <- ISSUE HERE
```

Why this is problematic / risk:

Ruff reports this as invalid syntax due to unexpected indentation. As written, the file will not parse correctly, which breaks tooling and prevents the module from being imported or executed reliably.

Corrected version of the full function:

```
def run_command(user_input):
    # Command Injection
    os.system("ls " + user_input)
    subprocess.call("echo " + user_input, shell=True)
```

Official documentation:

- Ruff invalid syntax rule: <https://docs.astral.sh/ruff/rules/invalid-syntax/>

2) Syntax error in `app.py` at `load\_pickle\_unsafe`

Finding: Ruff invalid-syntax — expected a statement.

Full function containing the issue:

```
def load_pickle_unsafe(filename):
    # Insecure Deserialization (High severity)
    with open(filename, 'rb') as f:
```

```
return pickle.loads(f.read())
```

Problematic line:

```
def load_pickle_unsafe(filename): # <- ISSUE HERE
```

Why this is problematic / risk:

Ruff reports the parser expected a statement at this location, which indicates the file structure is invalid. This prevents the code from being parsed cleanly and can block linting, testing, and execution.

Corrected version of the full function:

```
def load_pickle_unsafe(filename):  
    # Insecure Deserialization (High severity)  
    with open(filename, "rb") as f:  
        return pickle.loads(f.read())
```

Official documentation:

- Ruff invalid syntax rule: <https://docs.astral.sh/ruff/rules/invalid-syntax/>

3) Unused import in `vulnerable\_script.py`

Finding: Ruff F401 — imported but unused.

Full function containing the issue:

```
import sys  
  
from app import run_command, load_pickle_unsafe  
  
if __name__ == "__main__":  
    run_command(sys.argv[1])
```

Problematic line:

```
from app import run_command, load_pickle_unsafe # <- ISSUE HERE
```

Why this is problematic / risk:

`load_pickle_unsafe` is imported but never used. This adds noise, can confuse readers about intended behavior, and may indicate dead or incomplete code.

Corrected version of the full function/module section:

```
import sys  
  
from app import run_command
```

```
if __name__ == "__main__":  
    run_command(sys.argv[1])
```

Official documentation:

- Ruff F401 rule: <https://docs.astral.sh/ruff/rules/#f401-unused-import>

Teammate Comments Summary

No existing teammate review comments were provided.

Overall Risk Assessment

- **Risk Level:** HIGH
- **Reason:** The PR introduces hard-coded secrets and credentials flagged by SonarCloud, and Ruff also reports syntax issues that would prevent parts of the code from parsing correctly.